Route Prefix in ASP.NET Core Web API

What is route Prefix?

Route Prefix is a way to define a common URL starting point at the **controller level**. Once defined, every action method inside that controller automatically gets that prefix added to its URL.

Why do we need it?

To avoid writing the same word repeatedly in every route.

Problem:

Without Route Prefix (Repetition)

```
[ApiController]
public class BookController : ControllerBase
{
    [Route("book/all")]
    [HttpGet]
    public string GetAllBooks() { ... }

    [Route("book/{id}")]
    [HttpGet]
    public string GetBookById(int id) { ... }

    [Route("book/available")]
    [HttpGet]
    public string GetAvailableBooks() { ... }
}
```

N.B: The word “book” appears in EVERY route. If we want to change “book” to “library”, we must change it three times.

Solution:

With Route Prefix

```
[ApiController]
[Route("book")]              // ← Prefix defined ONCE at controller level
public class BookController : ControllerBase
{
    [Route("all")]           // ← Only unique part
    [HttpGet]
    public string GetAllBooks() { ... }

    [Route("{id}")]          // ← Only unique part
    [HttpGet]
    public string GetBookById(int id) { ... }

    [Route("available")]     // ← Only unique part
    [HttpGet]
    public string GetAvailableBooks() { ... }
}
```

Now: The word "book" appears ONLY ONCE. Change it to "library" in one place → all three URLs automatically become /library/all, /library/5, /library/available.

HOW ROUTE PREFIX WORKS

Controller Level	Action Method Level	Final
[Route("book")]	*[Route("all")]*	*/book/all*
[Route("book")]	*[Route("{id}")]*	*/book/5*
[Route("book")]	*[Route("available")]*	*/book/available*

The framework combines the controller prefix + the method route.

Now we are going to set up the project,

Project Setup

<u>Create the project:</u>

Command: dotnet new webapi -n LibraryApi4 -f net8.0

<u>Navigate to the project:</u>

Command: cd LibraryApi4

Install Packages

Before installing the packages, we should make sure that we are in the project folder 'LibraryApi4' in terminal. If not, then navigate to the exact project folder in terminal.

Command 1: dotnet add package Microsoft.EntityFrameworkCore --version 8.0.0

Command 2: dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.0

Command 3: dotnet add package Microsoft.EntityFrameworkCore.Tools --version 8.0.0

Command 4: dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.0

To check, whether the packages are successfully set up or not, we have to look over .csproj file,

LibraryApi4.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="8.0.24" />
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="8.0.0" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="8.0.0">
      <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive
      </IncludeAssets>
      <PrivateAssets>all</PrivateAssets>
    </PackageReference>
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>
```

Step 1: Create Model

Path: LibraryApi4/Models/Book.cs

Book.cs

Models > Book.cs > ...

```
namespace LibraryApi4.Models
{
    0 references
    public class Book
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Title { get; set; } = string.Empty;
        0 references
        public string Author { get; set; } = string.Empty;
        0 references
        public string Genre { get; set; } = string.Empty;
        0 references
        public int PublishedYear { get; set; }
        0 references
        public bool IsAvailable { get; set; }
        0 references
        public decimal Price { get; set; }
    }
}
```

Step 2: Create In-Memory Data (7 records)

Path: LibraryApi4/Data/BookData.cs

BookData.cs

Data > BookData.cs > ...

```
using LibraryApi4.Models;

namespace LibraryApi4.Data
{
    0 references
    public static class BookData
    {
        0 references
        public static List<Book> Books = new List<Book>
        {
            new Book {
                Id = 1,
                Title = "The Hobbit",
                Author = "J.R.R. Tolkien",
                Genre = "Fantasy",
                PublishedYear = 1937,
                IsAvailable = true,
                Price = 15.99m
            },
            new Book {
                Id = 2,
                Title = "1984",
                Author = "George Orwell",
                Genre = "Dystopian",
                PublishedYear = 1949,
                IsAvailable = false,
                Price = 12.99m
            },
            new Book {
                Id = 3,
                Title = "Pride and Prejudice",
                Author = "Jane Austen",
                Genre = "Romance",
                PublishedYear = 1813,
                IsAvailable = true,
                Price = 9.99m
            },
            new Book {
                Id = 4,
                Title = "The Great Gatsby",
                Author = "F. Scott Fitzgerald",
                Genre = "Classic",
                PublishedYear = 1925,
                IsAvailable = true,
                Price = 11.99m
            },
            new Book {
                Id = 5,
                Title = "Harry Potter",
                Author = "J.K. Rowling",
                Genre = "Fantasy",
                PublishedYear = 1997,
                IsAvailable = true,
                Price = 19.99m
            },
            new Book {
                Id = 6,
                Title = "Moby Dick",
                Author = "Herman Melville",
                Genre = "Adventure",
                PublishedYear = 1851,
                IsAvailable = false,
                Price = 14.99m
            },
            new Book {
                Id = 7,
                Title = "The Shining",
                Author = "Stephen King",
                Genre = "Horror",
                PublishedYear = 1977,
                IsAvailable = true,
                Price = 13.99m
            }
        };
    }
}
```

Step 3: Program.cs (Minimal)

Path: LibraryApi4/Program.cs

Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

Step 4: WITHOUT Route prefix

Path: LibraryApi4/Controllers/BookController.cs

```
BookController.cs ×
Controllers > BookController.cs > ...
using LibraryApi4.Data;
using LibraryApi4.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi4.Controllers
{
    [ApiController]
    0 references
    public class BookController : ControllerBase
    {
        // PROBLEM: "book" is repeated in every single route
        // If we rename to "Library",
        // we have to change ALL three routes

        [Route("book/all")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        [Route("book/{id}")]
        [HttpGet]
        0 references
        public ActionResult<Book> GetBookById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            if (book == null)
            {
                return NotFound();
            }
            return Ok(book);
        }

        [Route("book/available")]
        [HttpGet]
        0 references
        public ActionResult<IEnumerable<Book>> GetAvailableBooks()
        {
            var books = BookData.Books.Where(b => b.IsAvailable).ToList();
            return Ok(books);
        }
    }
}
```

These URLs work: /book/all, /book/5, /book/available

The Problem: The word “book” is written 3 times. Change to “library”? We must edit 3 lines.

Step 5: WITH Route prefix (The Solution)

Path: LibraryApi4/Controllers/LibraryController.cs

LibraryController.cs

Controllers > LibraryController.cs > ...

```
1   using LibraryApi4.Data;
2   using LibraryApi4.Models;
3   using Microsoft.AspNetCore.Mvc;
4
5   namespace LibraryApi4.Controllers
6   {
7       [ApiController]
8
9       // Step 1: Define the prefix ONCE at controller level
10      [Route("library")]
        0 references
11      public class LibraryController : ControllerBase
12      {
13          // Step 2: Each method only defines what comes AFTER the prefix
14          // Final URL = "library" + "all" = /library/all
15          [Route("all")]
16          [HttpGet]
            0 references
17          public ActionResult<IEnumerable<Book>> GetAllBooks()
18          {
19              return Ok(BookData.Books);
20          }
21
22          // Final URL = "library" + "{id}" = /library/5
23          [Route("{id}")]
24          [HttpGet]
            0 references
25          public ActionResult<Book> GetBookById(int id)
26          {
27              var book = BookData.Books.FirstOrDefault(b => b.Id == id);
28              if (book == null)
29              {
30                  return NotFound($"Book with {id} not found");
31              }
32              return Ok(book);
33          }
34
```

-

```
34
35          // Final URL = "library" + "available" = /library/available
36          [Route("available")]
37          [HttpGet]
            0 references
38          public ActionResult<IEnumerable<Book>> GetAvailableBooks()
39          {
40              var books = BookData.Books.Where(b => b.IsAvailable).ToList();
41              return Ok(books);
42          }
43
44          // Final URL = "library" + "genre/fantasy" = /library/genre/fantasy
45          [Route("genre/{genre}")]
46          [HttpGet]
            0 references
47          public ActionResult<IEnumerable<Book>> GetBooksByGenre(string genre)
48          {
49              var books = BookData.Books.Where(b => b.Genre
50                  .Equals(genre, StringComparison.OrdinalIgnoreCase)).ToList();
51              return Ok(books);
52          }
53      }
54  }
55
```

What Changed?

Before (No Prefix)	After (With Prefix)
[Route("library/all")]	[Route("library")] on controller + [Route("all")] on method
[Route("library/all")]	[Route("library")] on controller + [Route("{id}")] on method
[Route("library/available")]	[Route("library")] on controller + [Route("available")] on method

Benefit: Change “library” to “books” in ONE place (controller level) → ALL URLs update automatically.

Step 6: Overriding Route Prefix with Tilde (~)

What if we want ONE method to have a completely different URL, not using the prefix?

Example: Inside LibraryController, you want a health check at /health (NOT /library/health)

Solution: Use tilde ~ to ignore the controller prefix.

Add this method to,

Path: LibraryApi4/Controllers/LibraryController.cs

```
53
54          // TILDE (~) MEANS: Ignore the controller prefix
55          // Without ~ : URL would be /library/health
56          // With ~ : URL becomes /health
57          [Route("~/health")]
58          [HttpGet]
            0 references
59          public ActionResult HealthCheck()
60          {
61              return Ok(new
62              {
63                  Status = "API is running",
64                  Timestamp = DateTime.UtcNow
65              });
66          }
67      }
68  }
69
```

How Tilde Works:

Without Tilde	**With Tilde**
[Route("health")]	[Route("~/health")]
Result: /library/health	**Result:** /health
Prefix is Added	Prefix is IGNORED

Step 7: Cleaner Syntax (Recommended)

Instead of writing [Route("all")] [HttpGet], combine them into [HttpGet("all")].

Path: LibraryApi4/Controllers/StaffController.cs

StaffController.cs

Controllers > StaffController.cs > ...

```
using LibraryApi4.Data;
using LibraryApi4.Models;
using Microsoft.AspNetCore.Mvc;

namespace LibraryApi4.Controllers
{
    [ApiController]

    // Route prefix at controller level
    [Route("staff")]
    0 references
    public class StaffController : ControllerBase
    {
        // CLEANER: Route inside the HTTP method attributes
        // URL: GET /staff/all
        [HttpGet("all")]
        0 references
        public ActionResult<IEnumerable<Book>> GetAllBooks()
        {
            return Ok(BookData.Books);
        }

        // URL: GET /staff/5
        [HttpGet("{id}")]
        0 references
        public ActionResult<Book> GetBookById(int id)
        {
            var book = BookData.Books.FirstOrDefault(b => b.Id == id);
            if (book == null) return NotFound();
            return Ok(book);
        }
```

```
29
30          // URL: GET /staff/available
31          [HttpGet("available")]
            0 references
32          public ActionResult<IEnumerable<Book>> GetAvailableBooks()
33          {
34              var books = BookData.Books.Where(b => b.IsAvailable).ToList();
35              return Ok(books);
36          }
37
38          // URL: GET /staff/genre/fantasy
39          [HttpGet("genre/{genre}")]
            0 references
40          public ActionResult<IEnumerable<Book>> GetBooksByGenre(string genre)
41          {
42              var books = BookData.Books
43                  .Where(b => b.Genre
44                  .Equals(genre, StringComparison.OrdinalIgnoreCase))
45                  .ToList();
46              return Ok(books);
47          }
48
49          // Override with tilde - URL: GET /info/version
50          [HttpGet("~/info/version")]
            0 references
51          public IActionResult GetVersion()
52          {
53              return Ok(new { Version = "1.0.0",
54                  Api = "Library Management" });
55          }
56      }
57  }
58
```

Test our API – All Endpoints with Expected Responses

Let's build and run the application. And here we will use the command line or cli. We have to navigate to the exact project folder (LibraryApi4) and run those following commands,

Build the application,

Command: *dotnet build*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi4> dotnet build
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi4 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi4\bin\Debug\net8.0\LibraryApi4.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.69
```

To run the application,

Command: *dotnet run* (we usually get the URL from here to see application response).

We can run the project and can directly navigate to application response or Swagger UI.

Use can run the command,

Command: *dotnet watch run*

```
PS E:\PracticeProjects\DotnetApi\Routing\LibraryApi4> dotnet watch run
dotnet watch 🔥 Hot reload enabled. For a list of supported edits, see https://aka.ms/dotnet/hot-reload.
   💡 Press "Ctrl + R" to restart.
dotnet watch ⌚ Building...
  Determining projects to restore...
  All projects are up-to-date for restore.
  LibraryApi4 -> E:\PracticeProjects\DotnetApi\Routing\LibraryApi4\bin\Debug\net8.0\LibraryApi4.dll
dotnet watch 🚀 Started
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5165
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: E:\PracticeProjects\DotnetApi\Routing\LibraryApi4
```

LibraryApi4 1.0 OAS 3.0

http://localhost:5165/swagger/v1/swagger.json

Book

- GET /book/all
- GET /book/{id}
- GET /book/available

Library

- GET /library/all
- GET /library/{id}
- GET /library/available
- GET /library/genre/{genre}
- GET /health

Staff

- GET /staff/all
- GET /staff/{id}
- GET /staff/available
- GET /staff/genre/{genre}
- GET /info/version

TEST 1: Route Prefix Applied - /library/all

Request: GET http://localhost:5165/library/all

Response:


```
[{"id":1,"title":"The Hobbit","author":"J.R.R. Tolkien","genre":"Fantasy","publishedYear":1937,"isAvailable":true,"price":15.99},
{"id":2,"title":"1984","author":"George Orwell","genre":"Dystopian","publishedYear":1949,"isAvailable":false,"price":12.99},
{"id":3,"title":"Pride and Prejudice","author":"Jane Austen","genre":"Romance","publishedYear":1813,"isAvailable":true,"price":9.99},
{"id":4,"title":"The Great Gatsby","author":"F. Scott Fitzgerald","genre":"Classic","publishedYear":1925,"isAvailable":true,"price":11.99},
{"id":5,"title":"Harry Potter","author":"J.K. Rowling","genre":"Fantasy","publishedYear":1997,"isAvailable":true,"price":19.99},
{"id":6,"title":"Moby Dick","author":"Herman Melville","genre":"Adventure","publishedYear":1851,"isAvailable":false,"price":14.99},
{"id":7,"title":"The Shining","author":"Stephen King","genre":"Horror","publishedYear":1977,"isAvailable":true,"price":13.99}]
```


TEST 2: Route Parameter - /library/3

Request: GET http://localhost:5165/library/3

Response:


```
{"id":3,"title":"Pride and Prejudice","author":"Jane
Austen","genre":"Romance","publishedYear":1813,"isAvailable":true,"price":9.99}
```


TEST 3: Nested Route - /library/genre/fantasy

Request: GET http://localhost:5165/library/genre/fantasy

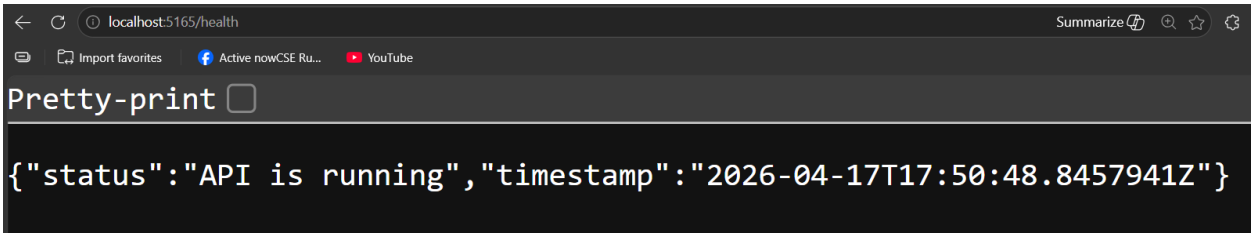
Response:


```
[{"id":1,"title":"The Hobbit","author":"J.R.R.
Tolkien","genre":"Fantasy","publishedYear":1937,"isAvailable":true,"price":15.99},
{"id":5,"title":"Harry Potter","author":"J.K.
Rowling","genre":"Fantasy","publishedYear":1997,"isAvailable":true,"price":19.99}]
```

TEST 4: Override with Tilde - /health

Request: GET http://localhost:5165/health

Response:



TEST 5: Staff Controller - /staff/all

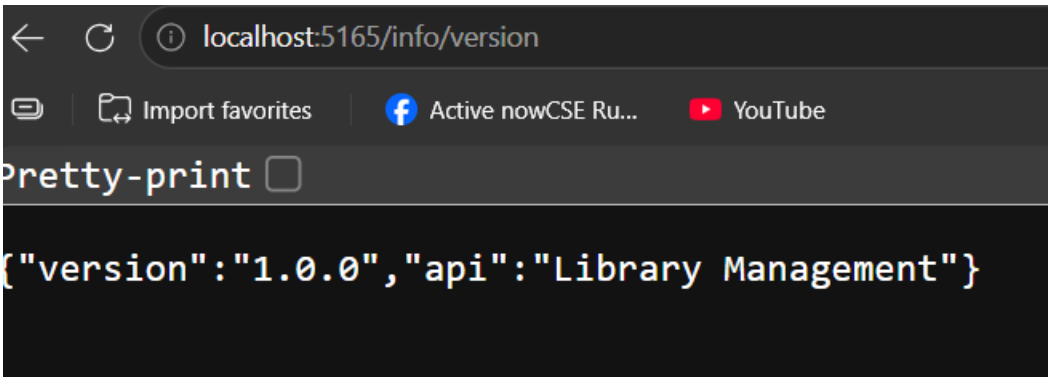
Request: GET http://localhost:5165/staff/all

Response:

TEST 6: Tilde in Staff Controller - /info/version

Request: GET http://localhost:5165/info/version

Response:



Key Takeaways

Concept	Explanation
Route Prefix	Common URL segment at controller level – applied to ALL actions
Inheritance	Every action method automatically gets prefix + its own route
Maintainability	Change prefix in ONE place → ALL routes update automatically
Tilde (~)	Overrides/ignores the controller-level for a specific action
Cleaner Syntax	Use [HttpGet("route")] instead of [Route("route")] [HttpGet]